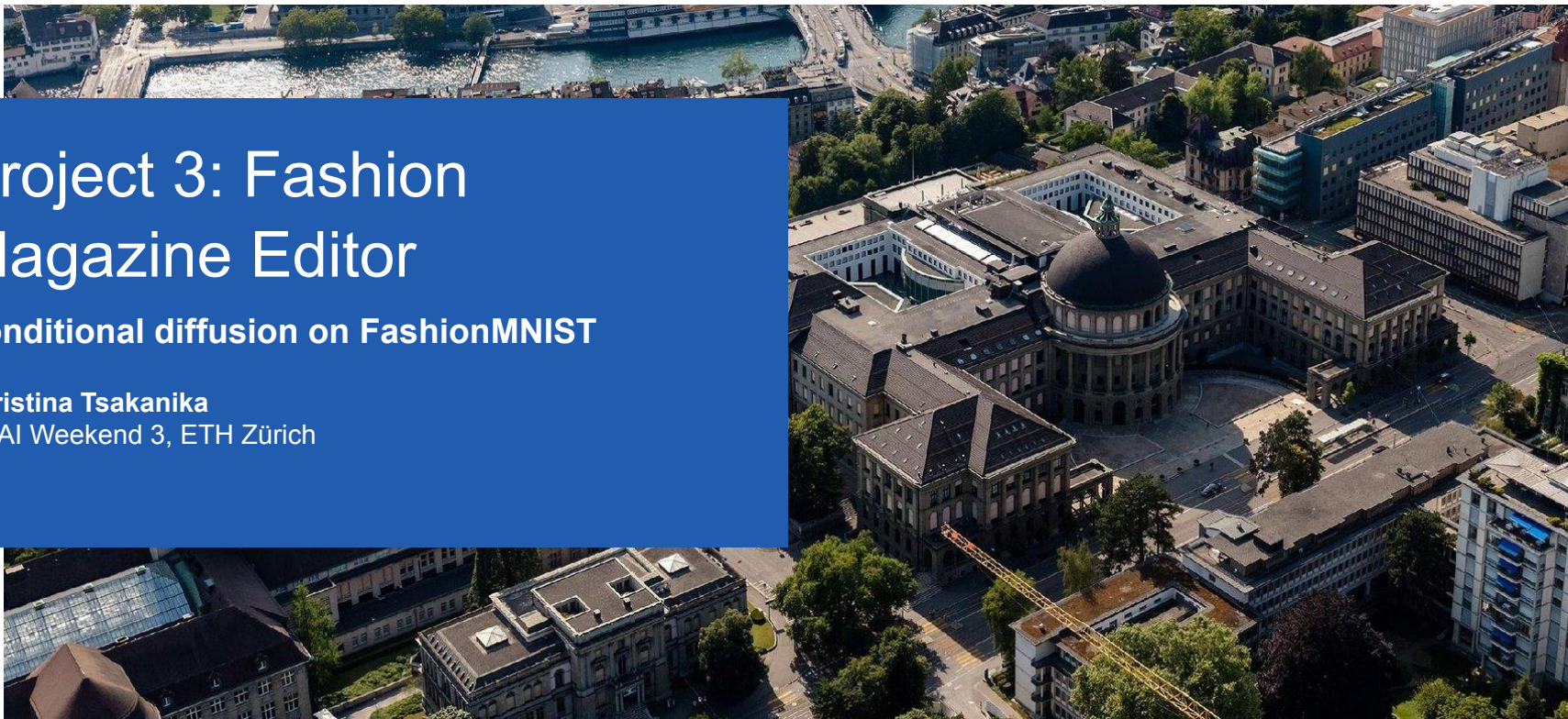


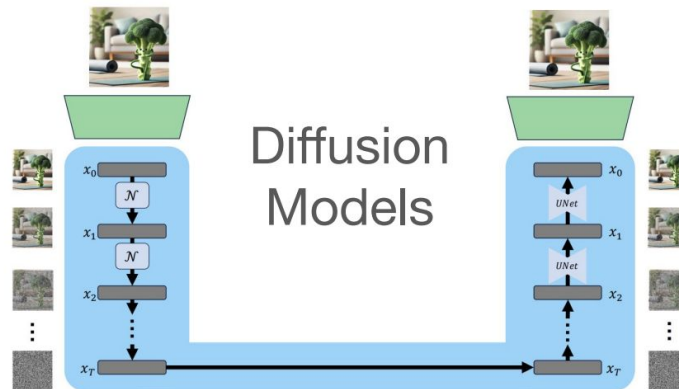
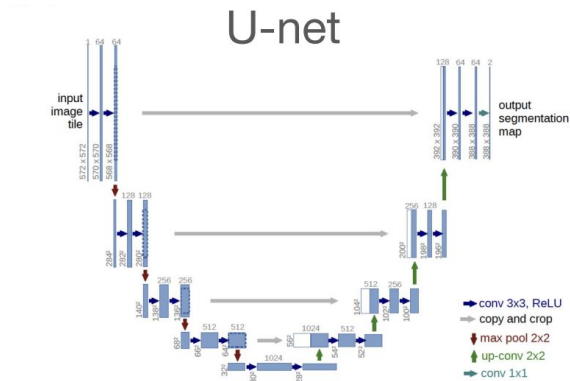
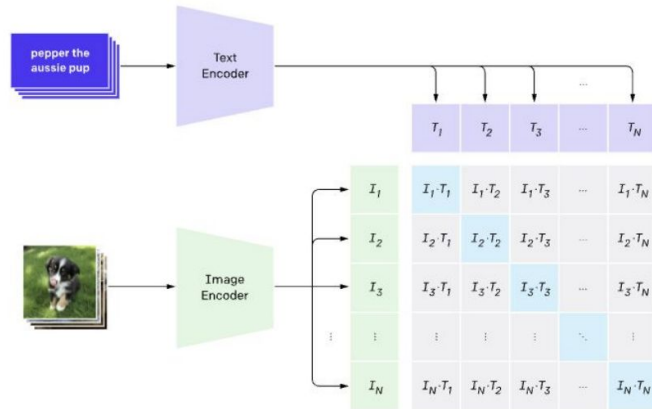
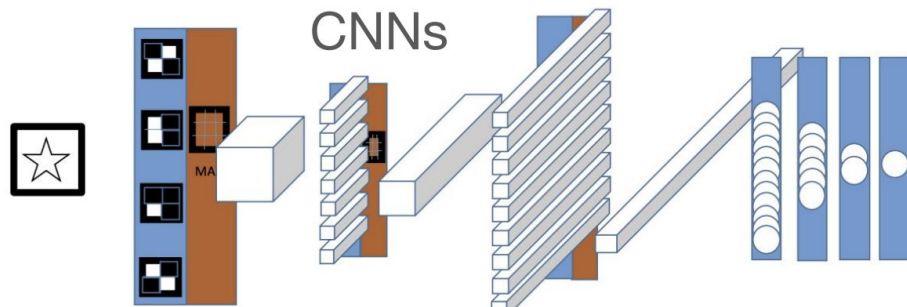
Project 3: Fashion Magazine Editor

Conditional diffusion on FashionMNIST

Christina Tsakanika
BMAI Weekend 3, ETH Zürich



You have learned a lot!



Project 3: Fashion Magazine Editor

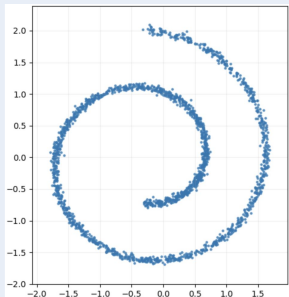
- Build a diffusion model from scratch that generates fashion items.
- Plug it into an app that writes a fashion magazine around the images.
- Dataset: FashionMNIST, 60,000 grayscale images, 10 clothing categories.

The ten categories



From the Swiss roll to images

What you already did



- Data: 10,000 points sampled from a 2D Swiss roll.
- Model: a small MLP taking the point and the timestep.
- Target: the noise sampled at that timestep.

What is new



- Data: images (32 x 32 grids of pixels)
- Model: UNet
- The output is now image shaped: one noise value per pixel.
- Generation is steered by a class label.

Conditional diffusion - Forward Process

No learning happens here. This is a fixed formula with precomputed constants.

1 Sample a timestep

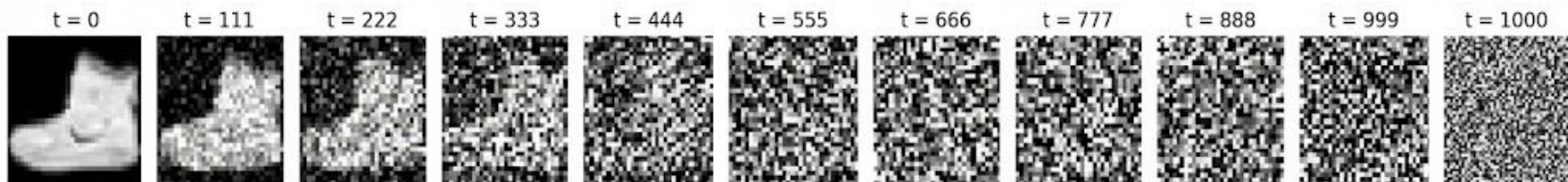
Draw t uniformly from 0 to $T-1$

2 Sample a noise pattern

Draw ϵ from a standard normal $\epsilon \sim \mathcal{N}(0, I)$

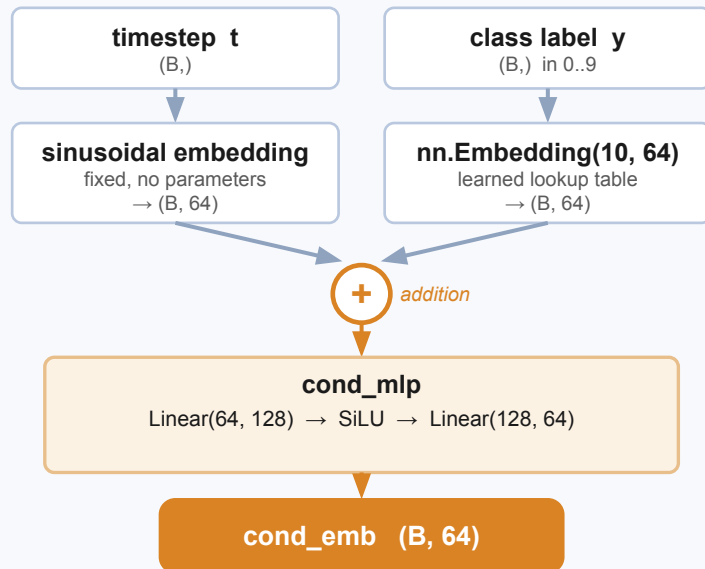
3 Build the noisy image

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$



Conditional diffusion - Train the UNet

Building the conditioning embedding



Pass both inputs to the UNet

1

1. the noisy image $x_\square$
2. the conditioning embedding built from t and the class label y

Compare prediction with target

2

- Mean squared error between the noise we sampled, ϵ , and the network output $\epsilon\theta$.

Conditional diffusion - Generation

1 Start from pure noise

2 Ask the trained UNet for the noise

Give it the current x_t together with the conditioning embedding for the class you want. UNet returns $\epsilon_\theta(x_t, t, y)$, an image shaped noise estimate.

3 Make a step back, then repeat

Feed that estimate into the reverse formula to get x_{t-1} , and repeat down to x_0 .

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t, y) \right) + \sigma_t z$$

ϵ_θ comes from the network. The $\sigma_t z$ term is dropped at the final step.

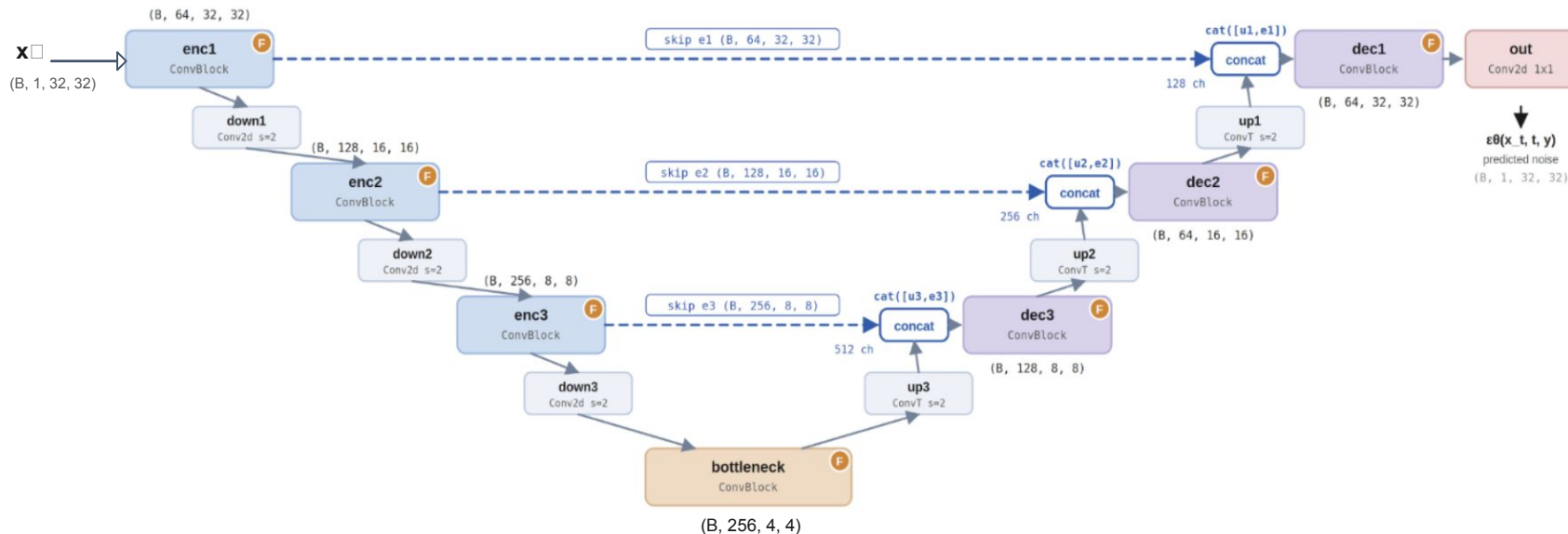
Class 0: T-shirt/top



The UNet you will implement

Input: the noisy image plus a conditioning embedding. Output: a noise tensor of exactly the same shape.

Encoder Bottleneck Decoder

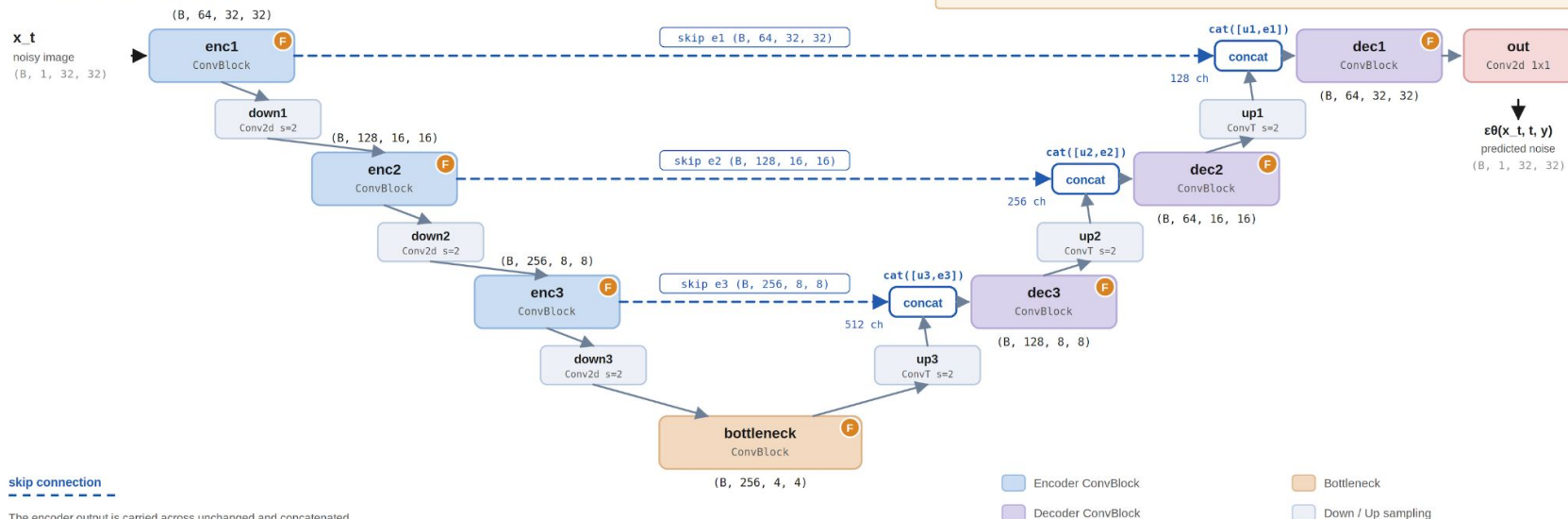


The UNet you will implement

1. Building the conditioning embedding



3. The U: encoder, bottleneck, decoder



2. Where the conditioning enters the network

F This badge marks every block that receives cond_emb.

All seven ConvBlocks get it: enc1, enc2, enc3, bottleneck, dec3, dec2, dec1.

Inside one ConvBlock

Two convolutions process the image $\rightarrow$ produce h

FiLM then rescales the result, one knob per channel:

$$h * (1 + \text{scale}) + \text{shift}$$

Your deliverables

Task 1

 2.1 Implement the forward noising function

Corrupt an image to any noise level t .

 3.1 Visualize the noising process

Task 2

4.1    Define the UNet

Encoder, bottleneck and decoder, with the conditioning embedding wired into every block.

Task 3

4.3    Train the UNet

Sample a random timestep per image, predict the noise, minimise the mean squared error.

Task 4

 5.2 Implement the denoising step

Implement the sampling equation and generate a garment of your chosen class from pure noise.

Reminder!

Change runtime type

Runtime type

Python 3

Hardware accelerator ?

☒ CPU ☐ H100 GPU ☐ G4 GPU ☐ A100 GPU

☐ L4 GPU ☐ T4 GPU ☐ v6e-1 TPU

☐ v5e-1 TPU

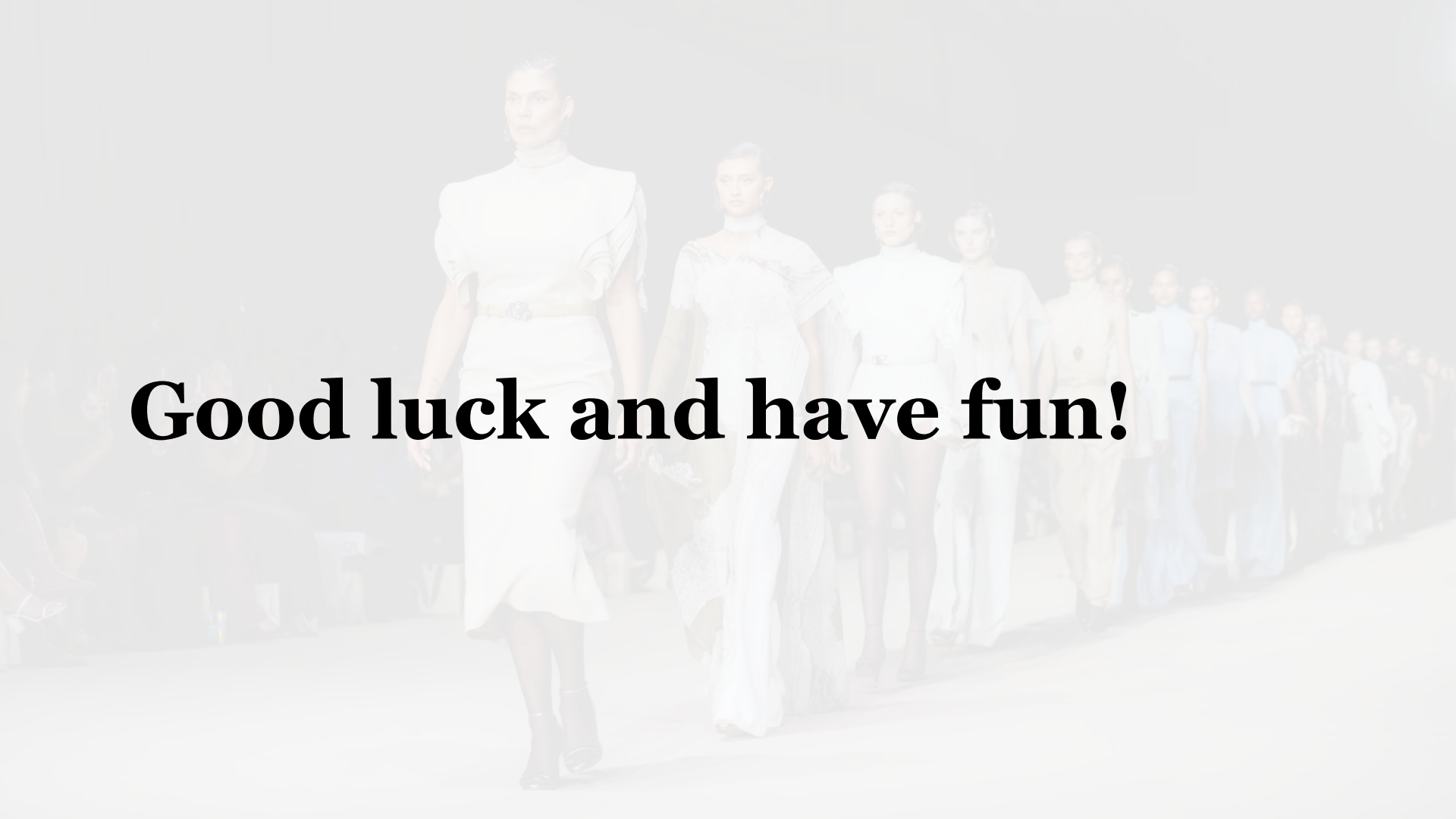
High-RAM ☐

Runtime version ?

Latest (recommended)

Cancel Save

L4 and T4 are the best for the scope of this project!



Good luck and have fun!

◆ Fashion *Generator*

BMAI-POWERED MAGAZINE CREATOR

ARTICLE TITLE

e.g. Spring Trends 2025

IMAGE STYLES

e.g. editorial high fashion, studio lighti



+ Add image style

ARTICLE DESCRIPTION

Describe in detail what the article should cover...

► Generate

🗑 Delete History

► SESSION HISTORY



Your article will appear here

Fill in the form and press Generate
to create your fashion article.

◆ Fashion *Generator*

BMAI-POWERED MAGAZINE CREATOR

ARTICLE TITLE

ETH Fashion Weekend 2

IMAGE STYLES

Spring cafe terrace, tables with noteb



+ Add image style

ARTICLE DESCRIPTION

Spring has arrived at ETH Zürich, and the engineering halls have never looked this good.

5-paragraph editorial article about ETH students who have quietly become the fashion icons of university life. Their personal style: layered linen, tailored trousers, clean sneakers. Profile the look: academic-core meets Swiss minimalism. Mention Polyterrasse as the runway, the main library as the backstage, and the coffee machine as the post-show party.

► Generate

🗑 Delete History

► SESSION HISTORY



Your article will appear here

Fill in the form and press Generate
to create your fashion article.

► Generate

🗑 Delete History

IMAGE 1

◆ EDIT IMAGE

Leave empty to keep, or describe
regeneration...

ARTICLE TEXT CHANGES

What would you like to change in the text?

🖋 Apply Feedback

- ✓ 🇫🇮 Image 1 CLIP score: 0.62 ✓
- ✓ Writing article...
- ✓ Assembling final HTML page...
- ✓ HTML generation attempt 1/3...
- ✓ ✓ HTML validated successfully.
- ✓ ✓ Done! Article saved as
article_20260303_150023.html
- ✓ ✓ Done!

► SESSION HISTORY

ARTICLE DESCRIPTION

Spring has arrived at ETH Zürich, and the engineering halls have never looked this good.

5-paragraph editorial article about ETH students who have quietly become the fashion icons of university life. Their personal style: layered linen, tailored trousers, clean sneakers. Profile the look: academic-core meets Swiss minimalism. Mention Polyterrasse as the runway, the main library as the backstage, and the coffee machine as the post-show party

ETH FASHION WEEKEND 2

Spring 2025 • Zürich Edition

The Quiet Revolution: How ETH Zürich Became Switzerland's Most Stylish Campus

By Alexandra Bernstein

Published Spring 2025

IMAGE STYLES

Spring cafe terrace, tables with notebooks

Spring has arrived at ETH Zürich, and the engineering halls have never looked this good. Paris and Milan guard their fashion week gates, a different kind of style revolution unfolding in the limestone corridors of Switzerland's premier technical university. The threads threading through the Polyterrasse aren't just solving differential equations and designing infrastructure—they're quietly rewriting the rules of academic dress. Forget the tired stereotypes of engineering students in rumpled hoodies and cargo pants. Today's ETH cohort has something far more complex than thermodynamics: the delicate equation of looking polished while pulling an all-nighter in the computational lab.



◆ Fashion *Generator*

BMAI-POWERED MAGAZINE CREATOR

IMAGE 1

◆ EDIT IMAGE

Leave empty to keep, or describe regeneration...

ARTICLE TEXT CHANGES

What would you like to change in the text?

✎ Apply Feedback

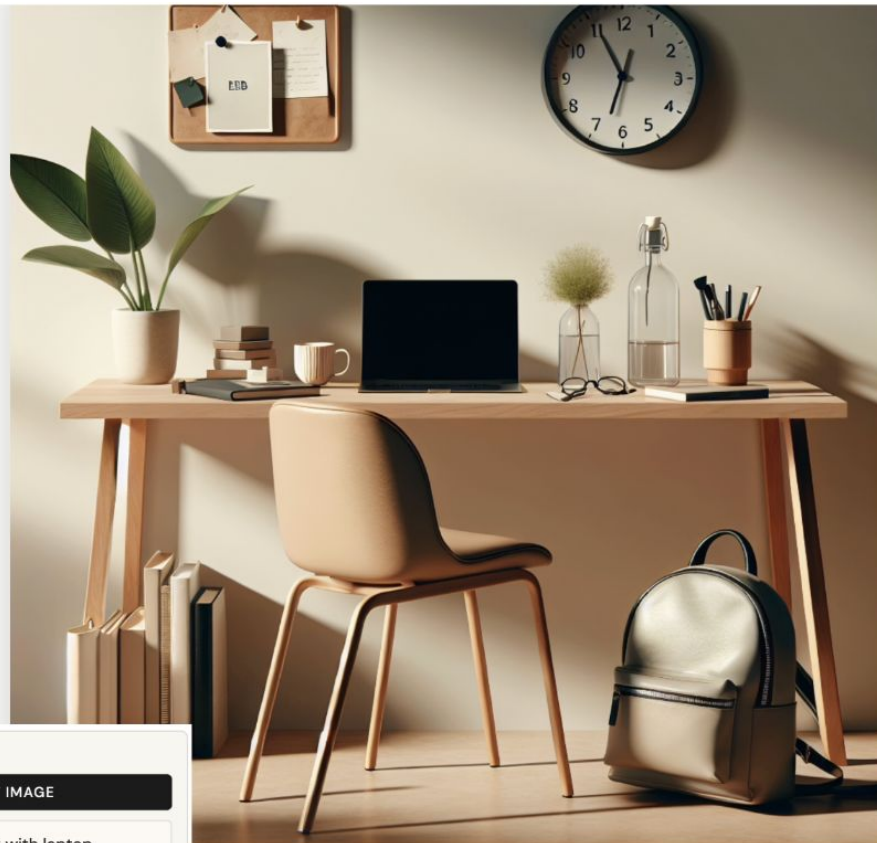
laptop, natural light...

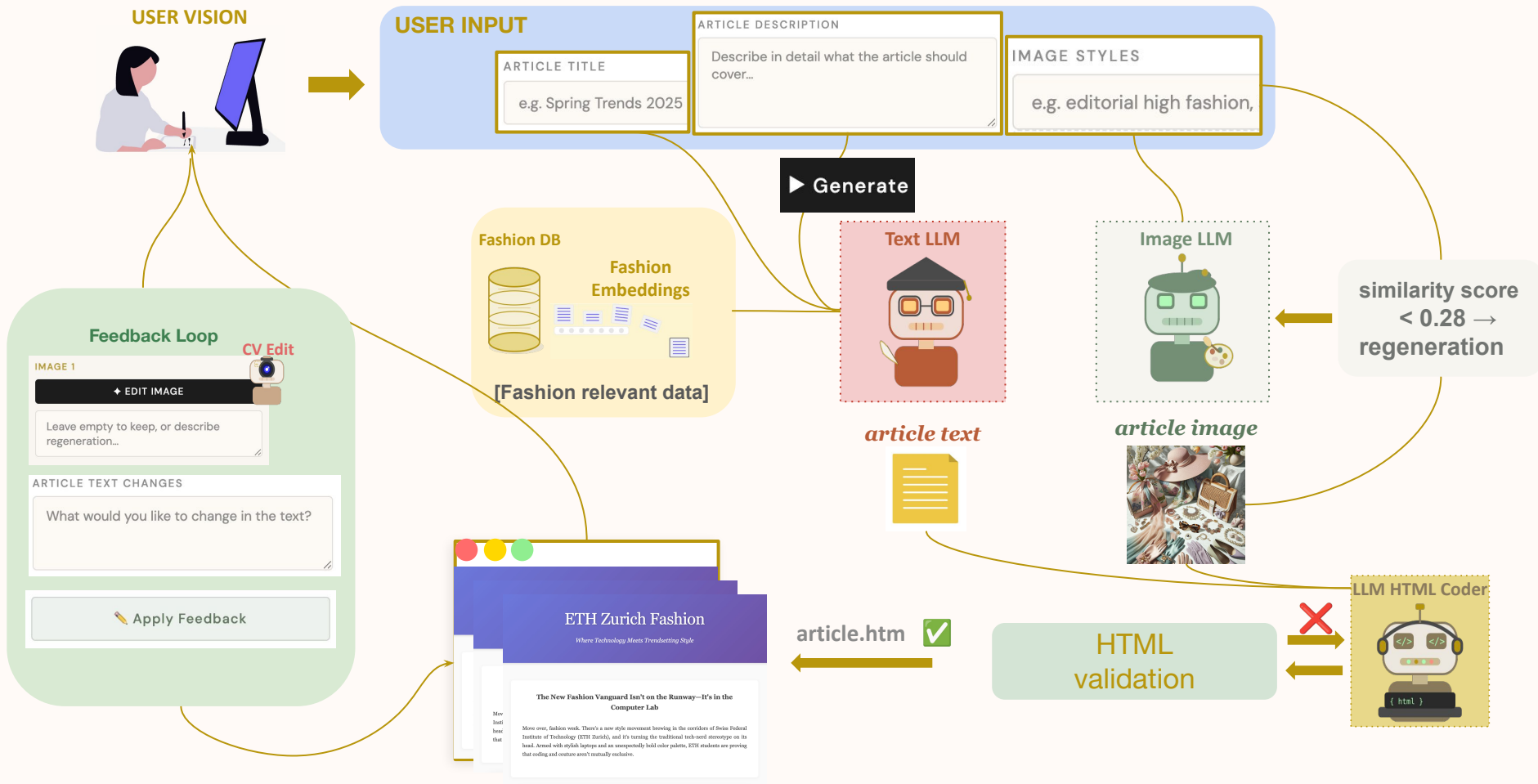
- ✓  Image 1 CLIP score: 0.66 
- ✓ Applying feedback to article...
- ✓ HTML generation attempt 1/3...
- ✓  HTML validated successfully.
- ✓  Feedback applied! Saved as article_20260303_151119_revised.html
- ✓  Done!

IMAGE 1

◆ EDIT IMAGE

Office Interior, desk with laptop.





STEP 1 — TWO SOURCE DATASETS



STEP 2 — KEEP DESCRIPTIONS ONLY

 **fashion_combined.csv**
~50K rows · one column: description · e.g. "Wool Jacket, Female, Brown, Geometric, Wool",
"Campus Women Navy Blue Mesh Running Shoes - Campus, A pair of navy blue running sports shoes, comes with Innovative upper technology
..."

STEP 3 — BERT TEXT EMBEDDINGS (computed once, stored offline)

 **bert_embeddings.npy**

50 K × 384

Text LLM



How do we use BERT?

bert_utils.py

Relevant dataset records

Text → Text · MiniLM encodes query and dataset rows into the same space



Article description query

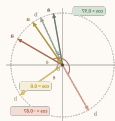
"spring workwear for young professionals"

Encode with BERT · all-MiniLM-L6-v2



```
_model.encode(descriptions, normalize_embeddings=True)
```

→ L2-norm built-in → 384-dim vector



Dot product vs 50 K stored embeddings

```
np.dot(EMBEDDINGS_NORM, query_vec)
```

Pre-normalised embeddings → cosine similarity = dot product

Top-5 fashion descriptions

injected as context into the article LLM

Text LLM



Text → Text

Both the article query and the 50 K stored records pass through the same text encoder path — so the dot product gives true cosine similarity

User**Title · Image Styles · Article Description****▶ Generate****Intent Embedding →
Relevant Dataset Records**

BERT text embed of article intent
Dot product vs. 50K stored embeddings
Top-5 matches injected as LLM context

ARTICLE DESCRIPTION

Describe in detail what the article should
cover...

**bert_embeddings.npy****Style Embeddings → Scoring**

CLIP text embed of each image style
Cosine similarity vs. generated image
Score ≥ 0.28 to accept · else retry

IMAGE STYLES

blue jeans

How do we use CLIP?

clip_utils.py

Image generation quality

Image → Text · cross-modal: does the image match the prompt?



Generated image

Style prompt:

red designer handbag

<code>vision_model → visual_projection</code>	<code>text_model → text_projection</code>
<code>→ 512-dim, L2-norm</code>	<code>→ 512-dim, L2-norm</code>



Cosine similarity

```
(img_norm * txt_norm).sum() # raw cosine ∈ [-1, 1]
```

```
score = (cosine + 1) / 2 # rescaled ∈ [ 0, 1]
```

score ≥ 0.28 ?

no ✗

yes ✓

Retry

Accept image

ARTICLE TEXT CHANGES

What would you like to change in the text?

IMAGE 1

✦ EDIT IMAGE

Leave empty to keep, or describe regeneration...

 Apply Feedback

How text feedback works

User describes what to change → LLM receives the full current HTML + the feedback instruction → patches only the relevant sections · images untouched

How image regeneration works

User changes style prompt for one image → DALL-E regenerates that image only → CLIP score check applied · other images unchanged

Going Further

The limit of text-to-image · why we need a CV pipeline

THE LIMIT



original

DALL-E regen

**Every regeneration replaces
the whole image**

pose · lighting · background — all different

Text-to-image models have no concept of 'this object'

THE SOLUTION



Detect

YOLOv8 identifies
objects in the image

clock 84%

chair 83%

vase 71%

tv 70%

potted plant 66%

laptop 57% ⚠️

Mask

User selects a label
binary mask generated

Edit

Only the masked region
is changed — rest preserved

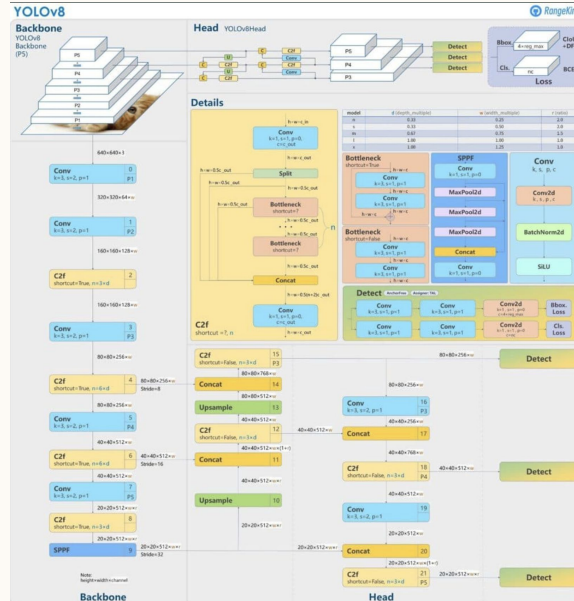
Same image — only the target region changes

identity · pose · lighting · background — all preserved

CV gives us spatial awareness · detect → mask → inpaint is the minimal pipeline to make targeted edits possible

Object Detection & Segmentation

Input Image

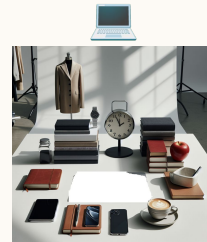


`result.masks.data:`
tensors representing
the shape of every
detected object

```

— Detected objects —
[0] laptop      confidence: 0.83
[1] cell phone   confidence: 0.83
[2] clock        confidence: 0.82
[3] cup          confidence: 0.65
[4] chair        confidence: 0.62

```



CV gives us spatial awareness · detect → mask → inpaint is the minimal pipeline to make targeted edits possible

Inpaint using OpenCV



Inpaint using Stable Diffusion

